

# 2025-CPL-exam

## 期末机试

时间：2026 年 1 月 5 日 18:00 ~ 21:00

|         |            |            |             |             |
|---------|------------|------------|-------------|-------------|
| 题目名称    | 心有灵犀       | 胡闹厨房       | 星系之心        | 优先队列        |
| 题目类型    | 传统型        | 传统型        | 传统型         | 传统型         |
| 目录      | hearts-one | overcooked | galaxy-core | prior-queue |
| 可执行文件名  | hearts-one | overcooked | galaxy-core | prior-queue |
| 每个测试点时限 | 1.0 秒      | 1.0 秒      | 0.5 秒       | 1.0 秒       |
| 内存限制    | 256 MiB    | 256 MiB    | 128 MiB     | 50 MiB      |
| 测试点数目   | 10         | 10         | 10          | 25          |
| 测试点是否等分 | 是          | 是          | 是           | 是           |

提交源程序文件名

|         |              |              |               |               |
|---------|--------------|--------------|---------------|---------------|
| 对于 C 语言 | hearts-one.c | overcooked.c | galaxy-core.c | prior-queue.c |
|---------|--------------|--------------|---------------|---------------|

编译选项

|         |                   |
|---------|-------------------|
| 对于 C 语言 | C Compile Options |
|---------|-------------------|

1. 考试时间：2026-01-05 18:00 - 21:00（左闭右开），账号为学号，密码为各自 oj 密码。考场的电脑时间可能有误，请以 OJ 显示的时间为准；
2. 持续 3 小时，共 4 道题，各题知识点已经在作业 OJ 中公布；
3. 如果 CLion 无法使用，请重启计算机进入 **win10-2 系统** 答题（CLion 已默认激活）。
4. 考试为闭卷，仅允许携带 **学生卡或身份证（必带，用于核验身份与签到）、水杯、无气味无声音零食、口罩、笔、餐巾纸**进入考场，监考老师会发放草稿纸，**禁止**携带任何电子设备（包括手表、手环、鼠标、键盘、耳机）与纸质材料进入考场；
5. 考试期间 **禁止**连接任何校外网（包括作业题解、熊佬博客），**禁止**使用南大云盘、自建服务器等校内服务，一经发现按期末考试作弊论处，无法访问课程网站，无法访问作业 OJ；
6. 考试期间 **禁止**查看、使用他人的代码，**禁止**登录他人账号，我们会检查登录地址，一经发现按期末考试作弊论处；
7. 可以提前交卷，请举手示意监考老师，离开前 **不要**关闭电脑，考试结束前 15 分钟禁止提前离场；
8. 若需要更多的草稿纸或遇到意外情况，请举手示意监考老师；
9. 任何离开考场的行为（如厕、沐浴更衣等）都需要经得监考老师同意，留下记录方可；

10. 考试结束时可将密码条和草稿纸带走，但 **不要** 关闭电脑；
11. 桌面上的 C 参考手册 (<https://ref.cpl.icu>) 与教材请自行探索；
12. 打开 CLion 时有概率跳出“不可信服务器的证书”的通知框，点击右下角“接受”即可；
13. 请 **经常性地**将代码在 **OJ 上提交**，以在线保存，防止电脑意外关机导致代码丢失。
14. 如果你在 `main` 函数中声明了过大的数组，可能会在本地无法运行。建议你在 `main` 函数之外声明这样的数组。

## 心有灵犀 (hearts-one)

### 【题目描述】

爬山回来后, YJ 和 BG 最近在玩一个“心有灵犀”的游戏。他们两人分别在纸上写下了一个正整数。为了证明他们的默契, 他们希望这两个数字的差值绝对值越小越好。

现在给出一组备选数字, 请你在其中选出两个不同的数, 使得它们的差值绝对值最小。

### 【输入格式】

输入共两行。

第一行包含一个正整数  $N$  ( $2 \leq N \leq 200,000$ )。

第二行包含  $N$  个互不相同的正整数  $a_i$ , 用空格隔开。  $0 < a_i \leq 10^8$ 。

### 【输出格式】

输出一个整数, 表示数组中任意两个不同元素之间差值绝对值的最小值。

### 【样例 1 输入】

```
1 6
2 10 50 12 1 100 13
```

### 【样例 1 输出】

```
1 1
```

### 【题目注解】

60% 的测试用例  $N \leq 1000$ 。

**60 分攻略:**

暴力遍历所有可能的数字组合, 找出最小值。

**100 分攻略:**

先排序, 再逐个计算相邻数字差值。建议使用 `qsort`, 冒泡排序的复杂度并不能达到本题要求。



图 1: 无论如何，完成了本学期课程的你辛苦了！

你在水里 扑通扑通 我 普通普通 你的一切 都值得歌颂  
扑通扑通 我 扑通扑通 多么地轻松  
你在心里 扑通扑通 我 扑通扑通 你的笑 照亮整个星空  
不懂不懂 我 不懂不懂 怎么能停止 这份悸动

## 胡闹厨房 (overcooked)

### 【题目描述】

小希正在联机游玩胡闹厨房！

在胡闹厨房中，有  $N$  种不同的食材，编号为 1 至  $N$  的整数；同时有  $M$  个待完成的订单，每个订单提供一份菜谱（食材的集合，一份菜谱内同种食材不会重复出现），需要按菜谱烹饪。

灶台上有一口大锅，初始为空。小希和他的队友会按顺序依次向锅内加入  $K$  份食材。每次放入食材后：

- **触发起火**：如果满足以下条件，大锅立刻起火，并清空锅内所有食材：
  - 加入食材后，当前锅内的食材集合不是任何一个未完成订单菜谱的子集。
- **触发烹饪**：如果满足以下条件，立刻进行烹饪操作，完成对应订单并清空锅内所有食材：
  - 加入食材后，当前锅内食材集合与某个未完成订单的菜谱完全相同（如有多个，任选一个）。
- **维持现状**：不满足上述两项，锅内保留当前食材，等待下一个食材加入。

题目注解中“题面及样例解释”部分有更详细的举例说明。

请你帮小希结算一场游戏。在结束时他能完成多少个订单？锅又会起火多少次？

### 【输入格式】

第一行三个整数  $N$ 、 $M$ 、 $K$ 。

接下来  $M$  行，每行包含若干个空格隔开的整数，描述一个订单的菜谱。每一行包括：

- 首先是一个整数  $size$ ，表示菜谱中食材的种类数；
- 接下来  $size$  个整数表示菜谱中的食材编号。

在同一个菜谱中，所有的食材编号互不相同，也就是一种食材只会出现最多一次。

接下来一行有  $K$  个空格隔开的整数，表示按顺序加入的所有食材。

### 【输出格式】

输出三行。

第一行输出游戏的结算评语：

- 如果所有订单都已完成，则输出 **Well Done!**；
- 如果没有完成所有订单，但大锅没有起火，则输出 **Not Bad!**；
- 否则输出 **Cooked!**。

第二行输出一个整数  $score$ ，表示完成的订单数。

第三行输出一个整数  $fire$ ，表示锅起火的次数。

**【样例 1 输入】**

```
1 2 4 7
2 1 2
3 1 1
4 1 2
5 1 1
6 1 2 2 2 2 2 2
```

**【样例 1 输出】**

```
1 Cooked!
2 3
3 4
```

**【样例 2 输入】**

```
1 3 6 9
2 1 1
3 3 3 1 2
4 1 1
5 1 3
6 2 1 3
7 1 2
8 2 3 2 3 1 1 1 3 1
```

**【样例 2 输出】**

```
1 Well Done!
2 6
3 0
```

**【样例 3 输入】**

```
1 4 4 8
2 3 2 1 4
3 3 1 4 2
4 3 1 4 3
```

```
5 1 2
6 4 3 1 2 2 4 1 4
```

**【样例 3 输出】**

```
1 Not Bad!
2 3
3 0
```

**【样例 4 输入】**

```
1 4 4 14
2 3 2 4 3
3 2 1 2
4 1 2
5 1 2
6 2 2 2 2 4 1 2 3 4 4 4 3 1 1
```

**【样例 4 输出】**

```
1 Cooked!
2 3
3 6
```

**【样例 5 输入】**

```
1 6 8 20
2 1 6
3 2 3 6
4 2 1 5
5 4 1 3 5 6
6 1 1
7 5 4 5 1 3 6
8 3 5 2 1
9 5 5 4 2 1 6
10 6 1 6 3 6 1 3 5 3 2 5 1 3 1 2 5 3 6 3 6
```

【样例 5 输出】

```
1 Cooked!  
2 5  
3 3
```

【数据范围与提示】

| 测试点编号  | $N, M$ 规模  | $K$ 规模      | $size_i$ 范围 | 特殊性质           |
|--------|------------|-------------|-------------|----------------|
| 1 ~ 3  | $\leq 200$ | $\leq 10^4$ | 1           | 所有订单都能完成，且从未起火 |
| 4 ~ 6  | $\leq 200$ | $\leq 10^4$ | 1           | 无              |
| 7 ~ 10 | $\leq 200$ | $\leq 10^4$ | $[1, 10]$   | 无              |

- 样例 1 符合测试点 4 ~ 6 的特征；
- 样例 2、3、4、5 符合测试点 7 ~ 10 的特征；

30 分攻略

直接根据特殊性质进行结算。

60 分攻略

鉴于  $size_i = 1$ ，每种菜谱只需要一种食材一份，可以先统计每种食材在未完成菜谱中的需求数（即出现在几个未完成菜谱中）。

每次加入食材时，检查该食材在未完成菜谱中的需求数：如果有需求，则完成一份订单，并更新需求数；否则，发生一次起火。

100 分攻略

按顺序加入所有食材，并维护锅中的食材集合。

加入食材时，先判断加入的食材是否与锅里已有的食材重合：

- 若是则起火；
  - 否则，将食材加入锅中，并依次对于所有还未完成订单的菜谱检查：
    - 如果锅中的每个食材都能在当前菜谱中找到：
      - \* 若锅内食材数和菜谱食材数相等，则进行烹饪，否则标记“继续烹饪”；
      - 如果锅中的某个食材不能在当前菜谱中找到，则无事发生；
- 检查完所有未完成订单的菜谱后，若无法“继续烹饪”，则起火。

【题面解释】

关于放入食材后的行为，举例如下。

现在有三份未完成菜谱，按顺序分别为 1 2 1 2 3 1 4；在只有一份食材 1 的大锅中：



- 若加入一份食材 1，加入后会触发起火，并清空锅内食材；
- 若加入一份食材 2，加入后会触发烹饪，按照菜谱 1 烹饪并清空锅内食材；
- 若加入一份食材 3，加入后会维持现状，无事发生；
- 若加入一份食材 4，加入后会触发烹饪，按照菜谱 3 烹饪并清空锅内食材；
- 若加入一份食材 5，加入后会触发起火，并清空锅内食材；

### 【样例 1 解释】

```
1 初始状态: N=2, M=4, K=7
2 订单 1: [2]
3 订单 2: [1]
4 订单 3: [2]
5 订单 4: [1]
6 -----
7 [步骤 1] 加入食材 1
8   -> 触发烹饪: 锅内 [1] 与订单 2 完全匹配。
9   -> 结果: 完成订单 2, 清空锅具。得分+1 (当前 1)。
10 [步骤 2] 加入食材 2
11   -> 触发烹饪: 锅内 [2] 与订单 1 完全匹配。
12   -> 结果: 完成订单 1, 清空锅具。得分+1 (当前 2)。
13 [步骤 3] 加入食材 2
14   -> 触发烹饪: 锅内 [2] 与订单 3 完全匹配。
15   -> 结果: 完成订单 3, 清空锅具。得分+1 (当前 3)。
16 [步骤 4] 加入食材 2
17   -> 触发起火: 锅内 [2] 不是任何未完成订单的子集。
18   -> 结果: 清空锅具。起火次数+1 (当前 1)。
19 [步骤 5] 加入食材 2
20   -> 触发起火: 锅内 [2] 不是任何未完成订单的子集。
21   -> 结果: 清空锅具。起火次数+1 (当前 2)。
22 [步骤 6] 加入食材 2
23   -> 触发起火: 锅内 [2] 不是任何未完成订单的子集。
24   -> 结果: 清空锅具。起火次数+1 (当前 3)。
25 [步骤 7] 加入食材 2
26   -> 触发起火: 锅内 [2] 不是任何未完成订单的子集。
```

27 -> 结果: 清空锅具。起火次数+1 (当前 4)。

28 -----

29 结算状态:

30 评语: Cooked!

31 完成订单: 3

32 起火次数: 4

### 【样例 2 解释】

1 初始状态: N=3, M=6, K=9

2 订单 1: [1]

3 订单 2: [1, 2, 3]

4 订单 3: [1]

5 订单 4: [3]

6 订单 5: [1, 3]

7 订单 6: [2]

8 -----

9 [步骤 1] 加入食材 2

10 -> 触发烹饪: 锅内 [2] 与订单 6 完全匹配。

11 -> 结果: 完成订单 6, 清空锅具。得分+1 (当前 1)。

12 [步骤 2] 加入食材 3

13 -> 触发烹饪: 锅内 [3] 与订单 4 完全匹配。

14 -> 结果: 完成订单 4, 清空锅具。得分+1 (当前 2)。

15 [步骤 3] 加入食材 2

16 -> 维持现状: 锅内 [2] 是订单 2 (及其他) 的子集。

17 -> 结果: 保留食材。

18 [步骤 4] 加入食材 3

19 -> 维持现状: 锅内 [2, 3] 是订单 2 (及其他) 的子集。

20 -> 结果: 保留食材。

21 [步骤 5] 加入食材 1

22 -> 触发烹饪: 锅内 [1, 2, 3] 与订单 2 完全匹配。

23 -> 结果: 完成订单 2, 清空锅具。得分+1 (当前 3)。

24 [步骤 6] 加入食材 1

25 -> 触发烹饪: 锅内 [1] 与订单 1 完全匹配。

26 -> 结果: 完成订单 1, 清空锅具。得分+1 (当前 4)。

27 [步骤 7] 加入食材 1

28 -> 触发烹饪: 锅内 [1] 与订单 3 完全匹配。

29 -> 结果: 完成订单 3, 清空锅具。得分+1 (当前 5)。

30 [步骤 8] 加入食材 3

31 -> 维持现状: 锅内 [3] 是订单 5 (及其他) 的子集。

32 -> 结果: 保留食材。

```
33 [步骤 9] 加入食材 1
34   -> 触发烹饪: 锅内 [1, 3] 与订单 5 完全匹配。
35   -> 结果: 完成订单 5, 清空锅具。得分+1 (当前 6)。
36 -----
37 结算状态:
38 评语: Well Done!
39 完成订单: 6
40 起火次数: 0
```

### 【样例 3 解释】

```
1 初始状态: N=4, M=4, K=8
2 订单 1: [1, 2, 4]
3 订单 2: [1, 2, 4]
4 订单 3: [1, 3, 4]
5 订单 4: [2]
6 -----
7 [步骤 1] 加入食材 4
8   -> 维持现状: 锅内 [4] 是订单 1 (及其他) 的子集。
9   -> 结果: 保留食材。
10 [步骤 2] 加入食材 3
11   -> 维持现状: 锅内 [3, 4] 是订单 3 (及其他) 的子集。
12   -> 结果: 保留食材。
13 [步骤 3] 加入食材 1
14   -> 触发烹饪: 锅内 [1, 3, 4] 与订单 3 完全匹配。
15   -> 结果: 完成订单 3, 清空锅具。得分+1 (当前 1)。
16 [步骤 4] 加入食材 2
17   -> 触发烹饪: 锅内 [2] 与订单 4 完全匹配。
18   -> 结果: 完成订单 4, 清空锅具。得分+1 (当前 2)。
19 [步骤 5] 加入食材 2
20   -> 维持现状: 锅内 [2] 是订单 1 (及其他) 的子集。
21   -> 结果: 保留食材。
22 [步骤 6] 加入食材 4
23   -> 维持现状: 锅内 [2, 4] 是订单 1 (及其他) 的子集。
24   -> 结果: 保留食材。
25 [步骤 7] 加入食材 1
26   -> 触发烹饪: 锅内 [1, 2, 4] 与订单 1 完全匹配。
27   -> 结果: 完成订单 1, 清空锅具。得分+1 (当前 3)。
28 [步骤 8] 加入食材 4
29   -> 维持现状: 锅内 [4] 是订单 2 (及其他) 的子集。
30   -> 结果: 保留食材。
```

```
31 -----
32 结算状态:
33 评语: Not Bad!
34 完成订单: 3
35 起火次数: 0
```

#### 【样例 4 解释】

```
1 初始状态: N=4,
2 订单 1: [2, 3, 4]
3 订单 2: [1, 2]
4 订单 3: [2]
5 订单 4: [2]
6 -----
7 [步骤 1] 加入食材 2
8   -> 触发烹饪: 锅内 [2] 与订单 3 完全匹配。
9   -> 结果: 完成订单 3, 清空锅具。得分+1 (当前 1)。
10 [步骤 2] 加入食材 2
11   -> 触发烹饪: 锅内 [2] 与订单 4 完全匹配。
12   -> 结果: 完成订单 4, 清空锅具。得分+1 (当前 2)。
13 [步骤 3] 加入食材 2
14   -> 维持现状: 锅内 [2] 是订单 1 (及其他) 的子集。
15   -> 结果: 保留食材。
16 [步骤 4] 加入食材 2
17   -> 触发起火: 锅内 [2, 2] 不是任何未完成订单的子集。
18   -> 结果: 清空锅具。起火次数+1 (当前 1)。
19 [步骤 5] 加入食材 4
20   -> 维持现状: 锅内 [4] 是订单 1 (及其他) 的子集。
21   -> 结果: 保留食材。
22 [步骤 6] 加入食材 1
23   -> 触发起火: 锅内 [1, 4] 不是任何未完成订单的子集。
24   -> 结果: 清空锅具。起火次数+1 (当前 2)。
25 [步骤 7] 加入食材 2
26   -> 维持现状: 锅内 [2] 是订单 1 (及其他) 的子集。
27   -> 结果: 保留食材。
28 [步骤 8] 加入食材 3
29   -> 维持现状: 锅内 [2, 3] 是订单 1 (及其他) 的子集。
30   -> 结果: 保留食材。
31 [步骤 9] 加入食材 4
32   -> 触发烹饪: 锅内 [2, 3, 4] 与订单 1 完全匹配。
33   -> 结果: 完成订单 1, 清空锅具。得分+1 (当前 3)。
```

```
34 [步骤 10] 加入食材 4
35   -> 触发起火: 锅内 [4] 不是任何未完成订单的子集。
36   -> 结果: 清空锅具。起火次数+1 (当前 3)。
37 [步骤 11] 加入食材 4
38   -> 触发起火: 锅内 [4] 不是任何未完成订单的子集。
39   -> 结果: 清空锅具。起火次数+1 (当前 4)。
40 [步骤 12] 加入食材 3
41   -> 触发起火: 锅内 [3] 不是任何未完成订单的子集。
42   -> 结果: 清空锅具。起火次数+1 (当前 5)。
43 [步骤 13] 加入食材 1
44   -> 维持现状: 锅内 [1] 是订单 2 (及其他) 的子集。
45   -> 结果: 保留食材。
46 [步骤 14] 加入食材 1
47   -> 触发起火: 锅内 [1, 1] 不是任何未完成订单的子集。
48   -> 结果: 清空锅具。起火次数+1 (当前 6)。
49 -----
50 结算状态:
51 评语: Cooked!
52 完成订单: 3
53 起火次数: 6
```

事实证明, 比起做菜, 小希 (和他的队友) 似乎更擅长纵火。



图 2: 游戏结束后小希正在和队友友好交流。

为我撩人 还为我双眸失神  
图我情真 还图我眼波销魂  
与我私奔 还与我做不贰臣  
夸我含苞待放 还夸我欲盖弥彰

## 星系之心 (galaxy-core)

### 【题目描述】

2077 年, 人类的足迹已经遍布银河系。天文学家发现, 只有结构稳定的「成熟星系」才可能孕育出高等文明。

星图被划分为  $N \times N$  的正方形网格地图。每个格子的数值为 0 或 1, 其中 1 表示该星系的物质, 0 表示虚空。

一个星系被定义为「成熟星系」, 当且仅当它关于其几何中心中心对称。

#### • 几何中心:

- 由该星系所有值为 1 的方格的  $x$  坐标范围  $[x_{min}, x_{max}]$  和  $y$  坐标范围  $[y_{min}, y_{max}]$  确定;
- 中心坐标为  $C_x = (x_{min} + x_{max})/2$ ,  $C_y = (y_{min} + y_{max})/2$ 。
- 本题保证若星系为成熟星系, 则  $C_x$  和  $C_y$  均为整数。

#### • 中心对称:

- 对于星系内的任意一个方格  $(x, y)$  (即值为 1 的格子), 其关于中心  $(C_x, C_y)$  的对称点  $(2C_x - x, 2C_y - y)$  的值也必须为 1。

在成熟星系内部, 存在一个被称为「星系之心」的区域, 它是该星系内部能够包含的 **最大正方形** 实心区域 (即该正方形内所有格子的数值都为 1)。

你的任务是:

1. 判断星图中的这个星系是否为成熟星系。
2. 如果是成熟星系, 计算出最大的「星系之心」的边长和几何中心坐标; 否则输出 0。

### 【输入格式】

第一行包含一个正整数  $T$ , 表示询问次数。

对于每次询问:

- 第一行包含一个正整数  $N$ , 表示正方形星图的行数和列数。
- 接下来  $N$  行, 每行一个由  $N$  个整数 (0 或 1) 组合而成的字符串, 表示星图数据。

请使用 `scanf("%s", ...)` 进行整行读入!

用 `%1d` 读入的话会慢出天际!

### 【输出格式】

对于每次询问, 输出一行:

- 如果星系是成熟星系, 输出三个整数: 最大的「星系之心」的边长、几何中心的  $x$  坐标、几何中心的  $y$  坐标 (以空格分隔)。
- 如果星系不是成熟星系, 或星图中没有任何星系物质 (全为 0), 输出 0。

**【样例 1 输入】**

```
1 6
2 5
3 11100
4 11100
5 11100
6 00000
7 00000
8 5
9 00100
10 01110
11 11111
12 01110
13 00100
14 3
15 110
16 111
17 001
18 1
19 1
20 3
21 000
22 010
23 000
24 5
25 11111
26 11111
27 11111
28 11111
29 11111
```

**【样例 1 输出】**

```
1 3 1 1
2 3 2 2
3 0
4 1 0 0
5 1 1 1
6 5 2 2
```

**【样例 2 输入】**

```
1 2
2 3
3 000
4 000
5 000
6 3
7 101
8 010
9 101
```

**【样例 2 输出】**

```
1 0
2 1 1 1
```

**【样例 3 输入】**

```
1 5
2 3
3 111
4 000
5 000
6 3
7 111
8 010
9 000
10 5
11 11111
12 11011
13 10001
14 11011
15 11111
16 3
17 101
18 000
19 000
20 7
21 0000000
```



```
22 0100010
23 1110111
24 1111111
25 1110111
26 0100010
27 0000000
```

【样例 3 输出】

```
1 1 1 0
2 0
3 2 2 2
4 1 1 0
5 3 3 3
```

【数据范围及约束】

基础分

只要能正确判断星系是否为成熟星系（即正确输出几何中心坐标或 0），即可获得该测试点 50% 的分数，无论「星系之心」的边长是否正确。

数据分布

「简单」星系：最大的「星系之心」的中心与星系几何中心重合（如样例 1 中的第 1、2 个询问）。

「复杂」星系：星系形状为回字形、哑铃形等，导致最大的「星系之心」的中心与星系几何中心不重合。

另外，复杂星系不保证连通（如样例 2 中的第 2 个询问、样例 3 中的第 3 个询问）。

| 测试点编号   | $N$ 规模      | $T$ 规模    | 星系形状特征   |
|---------|-------------|-----------|----------|
| 01 ~ 03 | $\leq 100$  | $\leq 20$ | 简单       |
| 04 ~ 05 | $\leq 100$  | $\leq 20$ | 复杂（混合简单） |
| 06 ~ 08 | $\leq 1000$ | $\leq 30$ | 简单       |
| 09 ~ 10 | $\leq 1000$ | $\leq 30$ | 复杂（混合简单） |

保证  $\sum N^2 \leq 1.2 \times 10^7$ 。

保证若星系为成熟星系，则其几何中心  $C_x$  和  $C_y$  均为整数。

**【样例 1 解释】**

- 第 1 个询问： $3 \times 3$  的正方形成熟星系，几何中心在  $(1,1)$ ，最大的「星系之心」的边长为 3；
- 第 2 个询问： $5 \times 5$  的菱形成熟星系，几何中心在  $(2,2)$ ，最大的「星系之心」的边长为 3（中心  $3 \times 3$  区域全为 1）；
- 第 3 个询问：L 形星系，不是成熟星系（不中心对称），输出 0；
- 第 4 个询问：最小星图（ $N = 1$ ），只有一个格子且为 1，几何中心在  $(0,0)$ ，最大边长为 1；
- 第 5 个询问：最大边长为 1 的成熟星系，中心只有一个 1，几何中心在  $(1,1)$ ，最大边长为 1；
- 第 6 个询问：最大边长等于  $N$  的成熟星系， $5 \times 5$  全为 1，几何中心在  $(2,2)$ ，最大边长为 5。

**【样例 2 解释】**

- 第 1 个询问：星图全为虚空，没有星系物质，输出 0；
- 第 2 个询问：X 形星系（不连通），是成熟星系，几何中心在  $(1,1)$ ，最大的「星系之心」的边长为 1。

**【样例 3 解释】**

- 第 1 个询问：中心在边框的成熟星系，第一行全为 1，几何中心在  $(1,0)$ （ $y = 0$  为星图边界），最大边长为 3；
- 第 2 个询问：伪中心在边框的不成熟星系，注意对称点不能越界，不中心对称，输出 0；
- 第 3 个询问：回字形成熟星系（复杂星系的典型例子），中心在  $(2,2)$ ，但中心区域为空心，最大的「星系之心」边长为 2；
- 第 4 个询问：离散的边框星系，两个 1 关于几何中心  $(1,0)$  对称，是成熟星系，最大边长为 1；
- 第 5 个询问：哑铃形成熟星系，中心在  $(3,3)$ ，最大的「星系之心」在两端，边长为 3。

**【提示】**

本题可以通过不同的策略获得部分分：

**10 分攻略**

1. 对于第一个测试点，所有询问的答案都是 0（非成熟星系或空星图）。
2. 直接输出  $T$  行 0 即可获得 10 分。

**55 分攻略（中心对称判定）**

1. 首先找出所有值为 1 的格子，记录它们的  $x$  坐标和  $y$  坐标的最小值和最大值。
2. 几何中心  $C_x = (x_{min} + x_{max})/2$ ， $C_y = (y_{min} + y_{max})/2$ 。

3. 遍历每个值为 1 的格子  $(x, y)$ ，检查其对称点  $(2C_x - x, 2C_y - y)$  是否也为 1。
4. 如果所有格子都满足对称条件，则为成熟星系；否则不是。
5. **注意：**一定要用判断保证你所检验的点都是星图内的，不会发生越界！
6. **提示：**几何中心也可以用所有值为 1 的格子的坐标平均值来计算（想一想，为什么这是等价的？）

### 75 分攻略（暴力搜索）

1. 对于  $N \leq 100$  的数据，可以使用四重循环暴力枚举。
2. 枚举每一个可能的正方形左上角  $(i, j)$  和边长  $L$ ，检查该区域是否全为 1。
3. 这种方法在小数据下是可行的，可以通过测试点 01 ~ 05。
4. 但是在规模较大的数据下会超时，请避免超时的发生，这会导致无法获得 50% 的基础分。
5. 若数据规模较大，则先跳过暴力搜索部分，直接输出基础分的结果（即只判断是否成熟，不计算最大边长）。

### 80 分攻略（中心扩散贪心）

1. 对于「简单」星系（最大正方形中心与几何中心重合），可以从几何中心开始，向外**一圈一圈**扩展扫描，直到遇到 0 为止。
2. 这种方法速度很快，但对于「复杂」星系（回字形/哑铃形）会得到错误答案。
3. 可以通过全部「简单」星系测试点。

### 满分攻略（二分答案 + 暴力验证）

1. 对于大规模数据，暴力枚举边长太慢，需要利用**单调性**进行优化。
2. **单调性：**如果存在边长为  $L$  的全 1 正方形，则一定存在边长为  $L - 1$  的全 1 正方形。因此可以对答案进行**二分查找**。
3. **二分框架：**在  $[0, N]$  的范围中，通过 `check(L)` 函数判断是否存在边长为  $L$  的合法正方形，二分查找最大的  $L$ 。
4. **check(L) 函数：**遍历整个星图，枚举所有可能的正方形左上角  $(i, j)$ ，检查该  $L \times L$  区域是否全为 1。

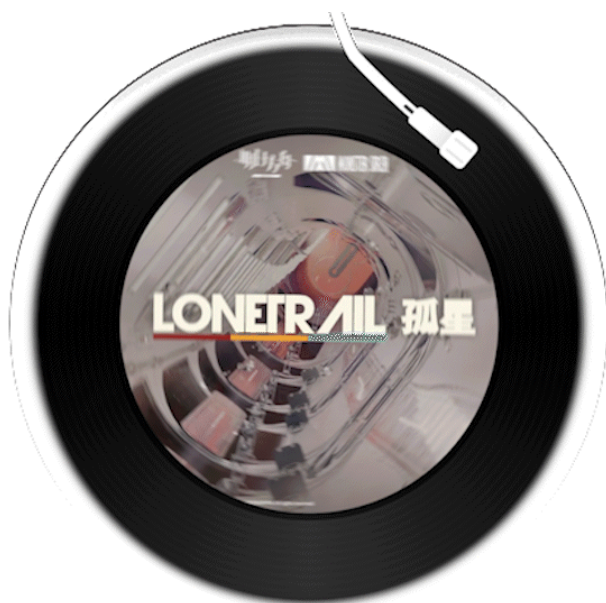


图 3: Ten! Nine! Eight! Seven! Six! Five! Four Three Two One! We have lifted off!

黑暗在 万家灯火之上  
我们却只对着银河仰望  
别紧闭 天生对峙暗的眼睛  
炙热的血才配得上光

# 优先队列 (prior-queue)

## 【题目描述】

Maya 正在排队求 Varecia 把 CPL 期末机试题目出得简单一点。突然一群小登涌来，插队到 Maya 的前面。Maya 很气愤，问他们为什么要插队。小登们亮出自己的小测分数，没有一个不超过 Maya 的。原来，这个队列是一个按小测分数排列的优先队列。没有考小测的 Maya 就这样被无情地挤到了队列的最后。

假设你正在管理一个优先队列，队列中的每一个元素有**唯一**的 ID（用  $id$  表示）和**优先级**（用  $p$  表示），不同的元素  $id$  不同， $p$  可能相同。你需要保证： $p$  大的元素总排在  $p$  小的元素前； $p$  相同的元素按入队先后顺序排列（也就是说，排序与  $id$  的数值并没有关系，我们也不会保证  $id$  按大小顺序给出）。

你需要为这个优先队列实现以下操作（ $n$  表示元素个数）：

| 操作名      | 实际操作 | 单次操作时间复杂度  |
|----------|------|------------|
| push     | 插入元素 | $O(1)$     |
| pop      | 弹出队首 | $O(p)$     |
| is_empty | 验证为空 | $O(1)$     |
| size     | 查看大小 | $O(1)$     |
| cut      | 截断队列 | $O(n + p)$ |
| print    | 打印元素 | $O(n + p)$ |

## 【输入描述】

共若干行。每行可能是以下几种指令之一：

- push  $id\ p$ ，表示将一个 ID 为  $id$ ，优先级为  $p$  的元素插入队列中；
- pop，表示将队列首个元素弹出；
- is\_empty，表示判断当前队列是否为空；
- size，表示查看当前队列元素个数；
- cut  $p$ ，表示将队列从  $p$  截断，即清除所有优先级小于  $p$  的元素；
- print  $n$ ，表示打印队列中按顺序遍历的第  $n$  个元素的 ID。

## 【输出描述】

共若干行。对每个下列指令，打印相应内容：

- is\_empty：打印一行：若队列为空，打印 true；否则，打印 false；
- size：打印一行，为当前队列的元素个数；
- print  $n$ ：打印一行，为当前队列按顺序遍历的第  $n$  个元素的 ID。

## 【样例 1 输入】

```
1 push 3 100
2 push 7 0
3 push 2 68
4 push 9 0
5 push 4 68
6 print 3
```

**【样例 1 输出】**

```
1 4
```

**【样例 2 输入】**

```
1 push 5 94
2 push 1 43
3 push 6 79
4 pop
5 push 7 29
6 push 3 57
7 pop
8 print 2
```

**【样例 2 输出】**

```
1 1
```

**【样例 3 输入】**

```
1 is_empty
2 size
3 push 9 25
4 push 2 44
5 is_empty
6 size
7 pop
8 pop
9 is_empty
10 push 4 1
11 push 7 37
```

```
12 push 0 18
13 size
14 is_empty
15 print 3
```

**【样例 3 输出】**

```
1 true
2 0
3 false
4 2
5 true
6 3
7 false
8 4
```

**【样例 4 输入】**

```
1 is_empty
2 push 6 38
3 push 7 68
4 cut 68
5 size
6 push 2 11
7 pop
8 push 0 38
9 push 4 47
10 print 2
11 is_empty
```

**【样例 4 输出】**

```
1 true
2 1
3 0
4 false
```

**【样例 5 输入】**

```
1 is_empty
2 push 5 48
3 push 7 12
4 push 6 37
5 push 3 77
6 push 1 98
7 size
8 print 2
9 pop
10 pop
11 pop
12 pop
13 pop
14 is_empty
15 push 9 37
16 push 8 37
17 push 0 37
18 push 4 37
19 push 2 37
20 size
21 print 4
22 cut 101
23 is_empty
```

**【样例 5 输出】**

```
1 true
2 5
3 3
4 true
5 5
6 4
7 true
```

**【数据范围及约束】**

请一定要仔细阅读后面的数据范围和解题提示，这有助于你拿到更多分数。

如果你获得了内存超限：



1. 在本题的限制下,  $100 \times 10^6$  的数组是开不出来的。
2. 使用后未释放的内存会一直保持占用状态, 直到程序结束运行为止。因此, 别忘了及时释放内存。

本题给出的 5 个样例分别对应了互斥的 20% 的测试点的特征。如果你没能通过样例, 请优先用它们调试。如果你使用了正确的解法通过了样例, 但仍然没能拿到相应的分数, 建议你检查一下数据范围。

### 一、操作类型

- 对 20% 的测试点, 只含有 `push`、`print` 操作;
- 对另外 20% 的测试点, 只含有 `push`、`pop`、`print` 操作;
- 对另外 20% 的测试点, 只含有 `push`、`pop`、`is_empty`、`size`、`print` 操作;
- 对剩余 40% 的测试点, 含有全部 6 种操作。

由于操作类型比较多, 强烈建议你按以上顺序实现各部分操作, 完成每部分操作之后立即提交, 拿到相应的分数, 确保正确后再继续写后面的操作。

### 二、元素个数和操作次数

- 对 60% 的测试点, 任何时刻队列中的元素个数不大于  $10^4$ , `push` 操作次数不大于  $2.5 \times 10^4$ , 总操作次数不大于  $5 \times 10^4$
- 对剩余 40% 的测试点, 任何时刻队列中的元素个数不大于  $10^6$ , `push` 操作次数不大于  $2.5 \times 10^6$ , 总操作次数不大于  $5 \times 10^6$ ;

### 三、其他

- 对 100% 的测试点,  $id$  为整数,  $0 \leq id \leq 2^{31} - 1$ ;
- 对 100% 的测试点,  $p$  为整数,  $0 \leq p \leq 100$ ; 但可能出现 `cut 101`, 也就是清除所有元素。
- 对 100% 的测试点, 保证 `pop` 时  $size > 0$ ; `print n` 中  $0 < n \leq size$  ( $size$  表示当前队列大小)。

### 【提示】

#### 60 分写法

用略长于  $10^6$  的 `int` 数组存储队列中所有的元素, `push` 时遍历寻找插入位置, 再移动之后的元素; `pop` 时删除首个元素, 再移动之后的元素。每次操作都可能移动大量的元素, 所以这样写会获得时间超限。

有一些“更高端的”数组写法可以拿到 100 分, 所以我们不会在考后人工把数组写法还原为 60 分。

## 100 分写法

注意到一个很有用的信息： $p$  的值有固定的范围，而且大小比  $n$  要小得多。这么看来，似乎可以将元素们以  $p$  值大小为索引分组存储。

如下图：把带头尾的单链表装进数组里，数组的下标即为该链表中元素的  $p$  值。由于  $p$  足够小，寻找其最值时直接遍历即可。这样的实现就足够通过本题。这个实现又被叫做桶式优先队列。你还记得桶计数法吗？

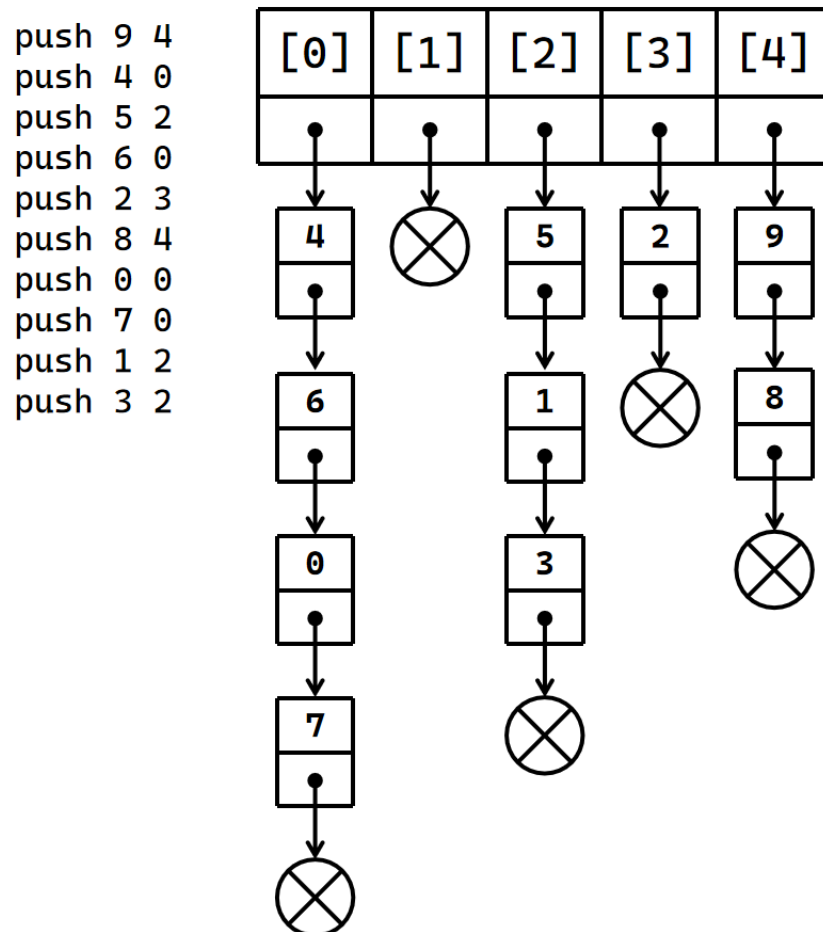


图 4：桶式优先队列



图 5: 被挤到队尾的 Maya

你们都是对的 他说拜托 让开一下  
让我大醉一场 殿堂之上 背闲诗两三章  
让我穿白衣裳 过疯人巷 去泥潭捞月光  
望大人们见谅 提点的话我懂也不想装  
世人寻黄金乡 我找月亮 何必同往  
话谢过 路借过 你是你 我是我  
话谢过 路借过 你是你 我是我